

「2026년도 2학기 한국성서대학교 AI융합학부」

# AIoT프로그래밍

아두이노 출력모듈: 세그먼트LED, LCD

4주차



# 목차

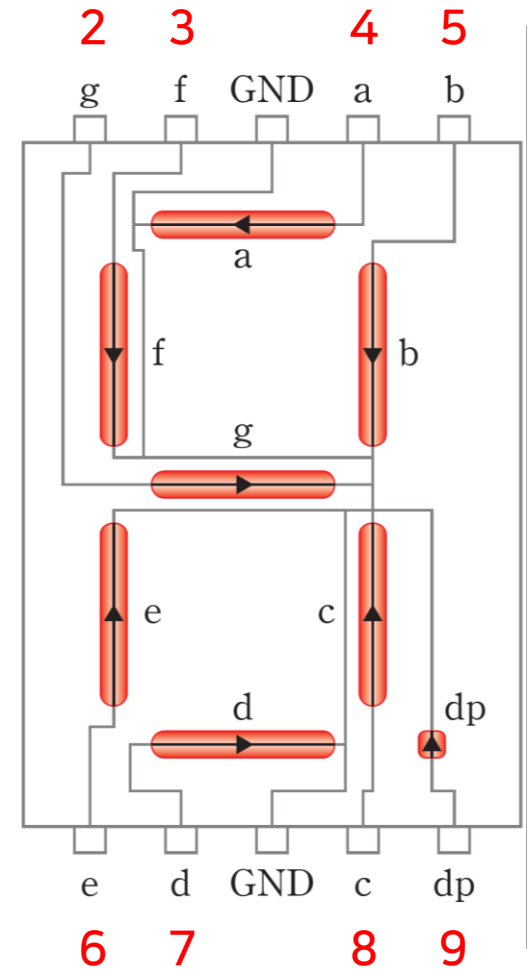
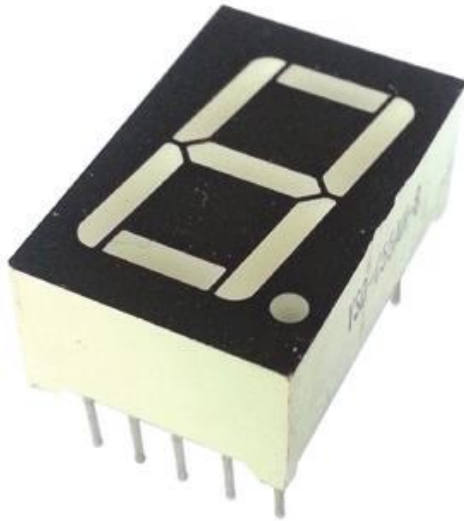
1. 세그먼트 LED
2. LCD

# 01 세그먼트 LED

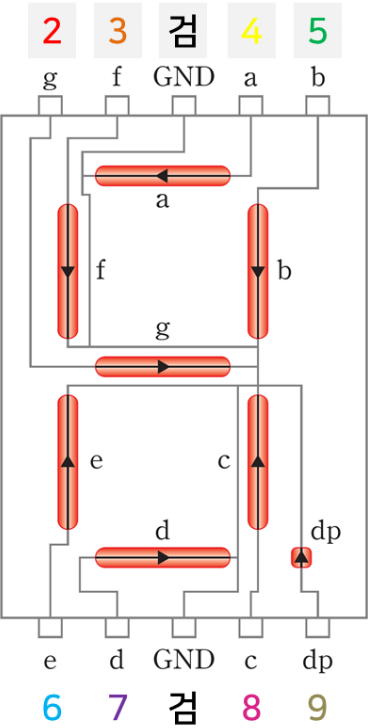
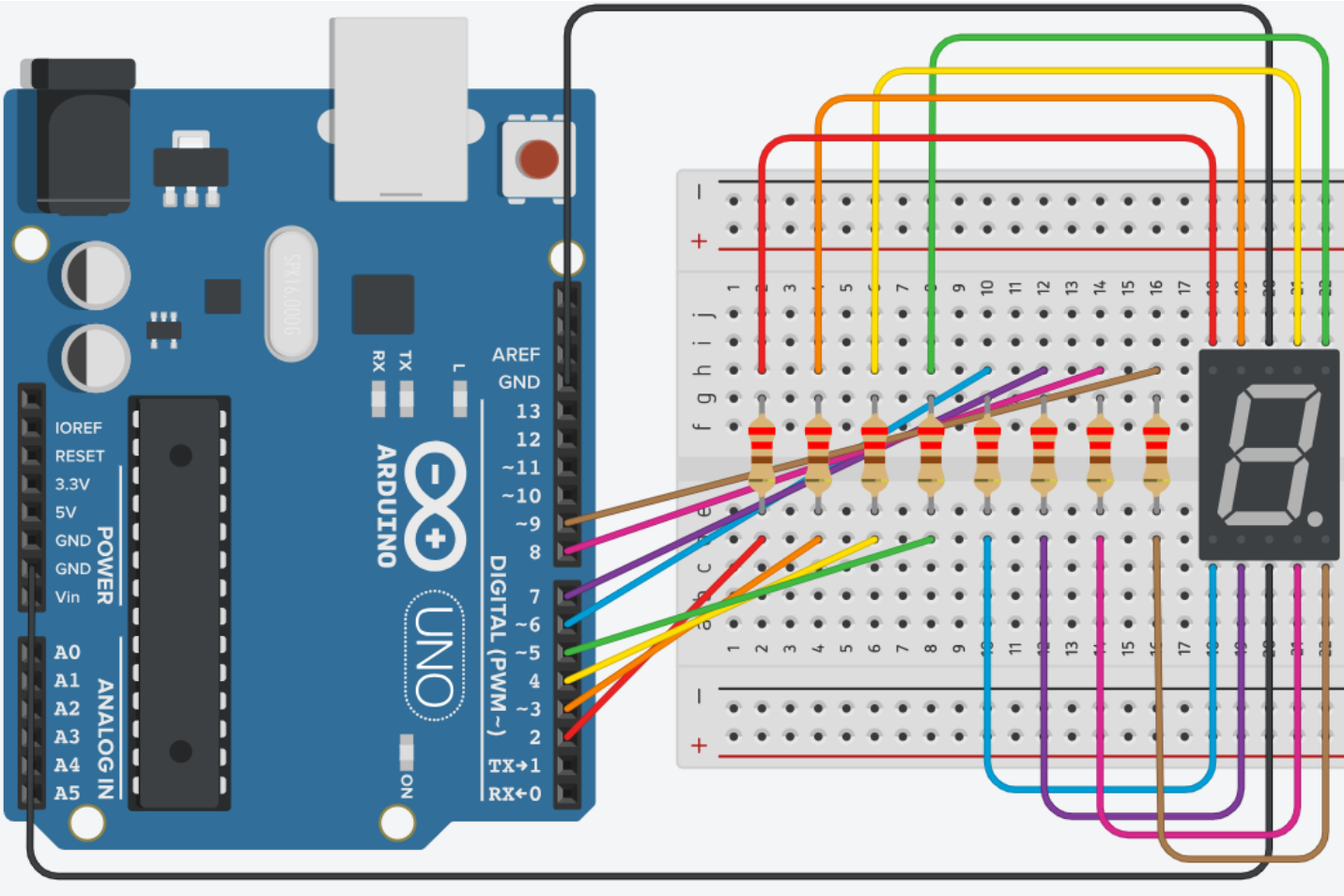
# 세그먼트 표시 장치

## ■ 8개의 LED로 문자, 숫자 등 표시

- 문자 표시를 위한 7개의 세그먼트 + 소수점 1개
- 공통핀과 제어핀으로 구성됨
- 디지털 출력



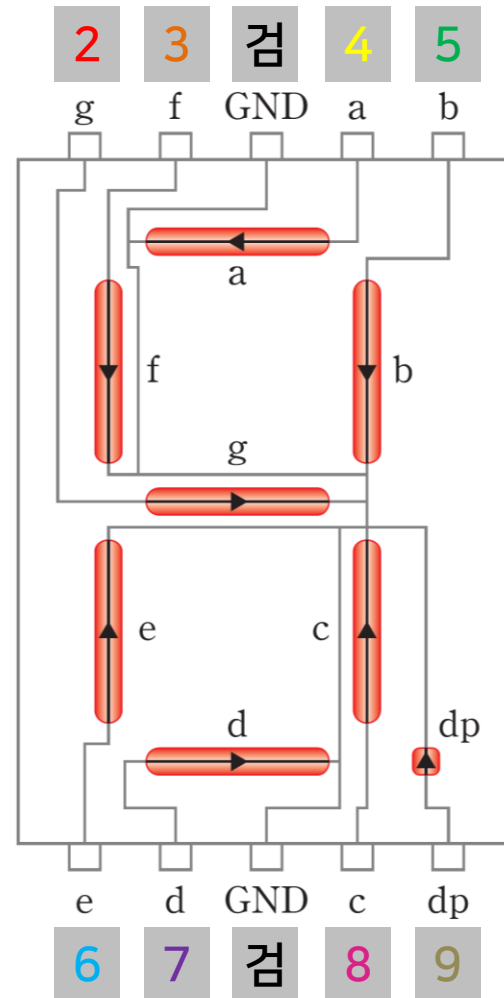
# 세그먼트 회로



# uno\_ex05\_01: 세그먼트 제어

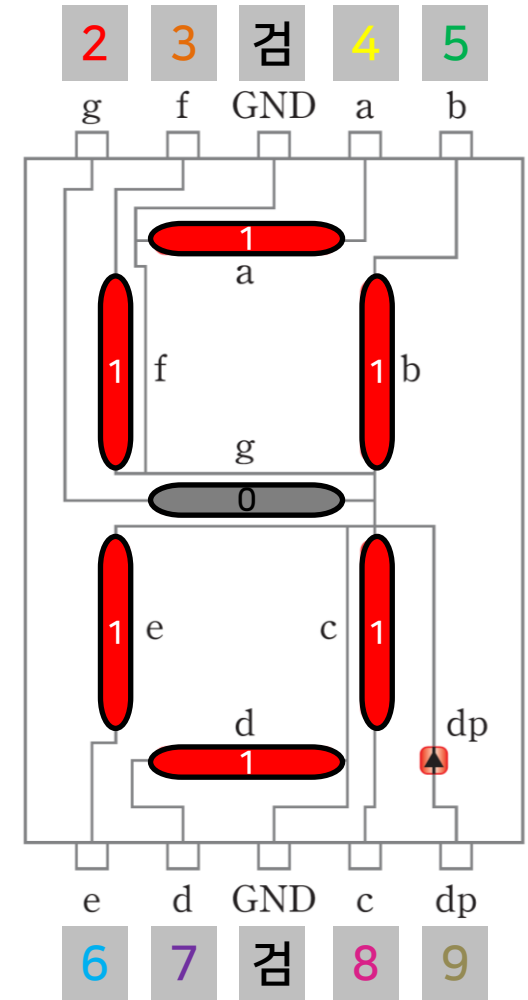
반드시 하나씩 확인!!!

```
1  const unsigned int led_g = 2;
2  const unsigned int led_f = 3;
3  const unsigned int led_a = 4;
4  const unsigned int led_b = 5;
5  const unsigned int led_e = 6;
6  const unsigned int led_d = 7;
7  const unsigned int led_c = 8;
8  const unsigned int led_dp = 9;
9
10 void setup() {
11     // led 초기화
12     pinMode(led_a, OUTPUT);
13     pinMode(led_b, OUTPUT);
14     pinMode(led_c, OUTPUT);
15     pinMode(led_d, OUTPUT);
16     pinMode(led_e, OUTPUT);
17     pinMode(led_f, OUTPUT);
18     pinMode(led_g, OUTPUT);
19     pinMode(led_dp, OUTPUT);
20
21     // led 켜기
22     digitalWrite(led_a, HIGH);
23     digitalWrite(led_b, HIGH);
24     digitalWrite(led_c, HIGH);
25     digitalWrite(led_d, HIGH);
26     digitalWrite(led_e, HIGH);
27     digitalWrite(led_f, HIGH);
28     digitalWrite(led_g, HIGH);
29     digitalWrite(led_dp, HIGH);
30
31 }
32
33
34 void loop() {}
```



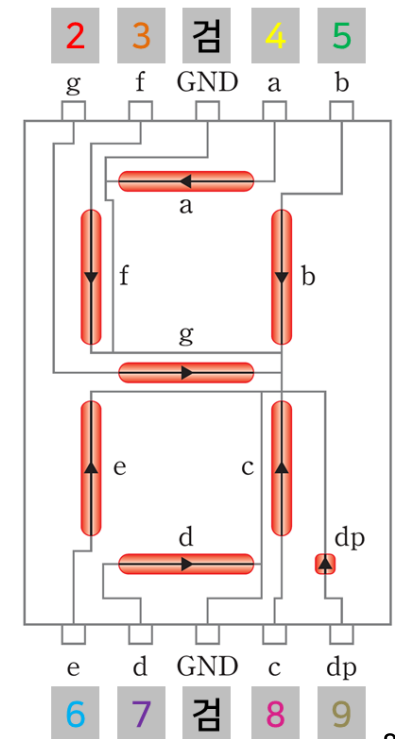
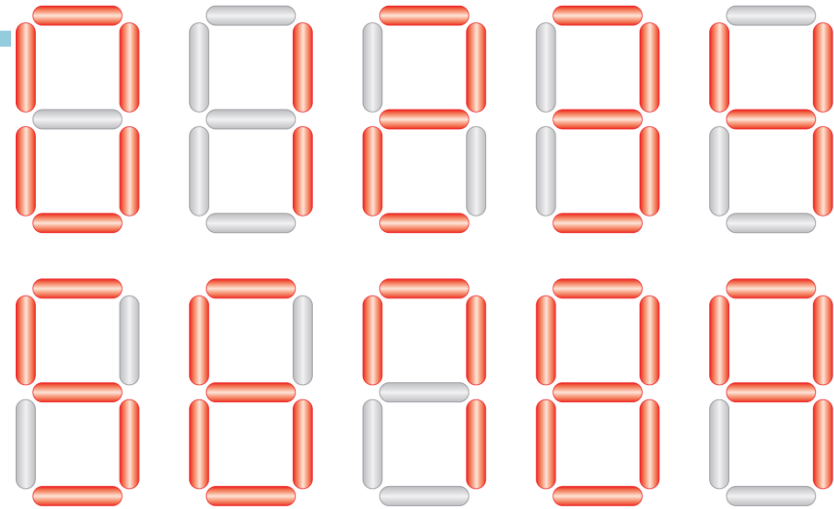
## uno\_ex05\_02: '0' 문자 출력

```
1  const int seg[] = {2,3,4,5,6,7,8}; //소수점 제외
2  const int num0[] = {0,1,1,1,1,1,1};
3
4
5  void setup() {
6    for(int i=0; i<7; i++) pinMode(seg[i], OUTPUT);
7  }
8
9  void loop() {
10   for(int i=0; i<7; i++) digitalWrite(seg[i], num0[i]);
11   delay(500);
12   for(int i=0; i<7; i++) digitalWrite(seg[i], 0);
13   delay(500);
14 }
```



# uno\_ex05\_03: 0~9 문자 배열에 저장하고 출력

```
1  const int seg[] = {2,3,4,5,6,7,8}; //소수점 제외
2  const int numPattern[10][7] = {
3      {0,1,1,1,1,1,1}, //문자 0 패턴
4      {0,0,0,1,0,0,1}, //문자 1 패턴
5      {1,0,1,1,1,1,0}, //문자 2 패턴
6      {1,0,1,1,0,1,1}, //문자 3 패턴
7      {1,1,0,1,0,0,1}, //문자 4 패턴
8      {1,1,1,0,0,1,1}, //문자 5 패턴
9      {1,1,1,0,1,1,1}, //문자 6 패턴
10     {0,1,1,1,0,0,1}, //문자 7 패턴
11     {1,1,1,1,1,1,1}, //문자 8 패턴
12     {1,1,1,1,0,0,1} //문자 9 패턴
13 };
14
15
16 void setup() {
17     for(int i=0; i<7; i++) pinMode(seg[i], OUTPUT);
18 }
19
20 void loop() {
21
22     for(int j=0; j<10; j++){
23         for(int i=0; i<7; i++) digitalWrite(seg[i], numPattern[j][i]);
24         delay(1000);
25         for(int i=0; i<7; i++) digitalWrite(seg[i], 0);
26     }
27 }
28 }
```



# uno\_ex05\_04: 사용자 입력 받아서 해당 문자 출력

```
1  const int seg[] = {2,3,4,5,6,7,8}; //소수점 제외
2  const int numPattern[10][7] = {
3      {0,1,1,1,1,1,1}, //문자 0 패턴
4      {0,0,0,1,0,0,1}, //문자 1 패턴
5      {1,0,1,1,1,1,0}, //문자 2 패턴
6      {1,0,1,1,0,1,1}, //문자 3 패턴
7      {1,1,0,1,0,0,1}, //문자 4 패턴
8      {1,1,1,0,0,1,1}, //문자 5 패턴
9      {1,1,1,0,1,1,1}, //문자 6 패턴
10     {0,1,1,1,0,0,1}, //문자 7 패턴
11     {1,1,1,1,1,1,1}, //문자 8 패턴
12     {1,1,1,1,0,0,1} //문자 9 패턴
13 };
14
15 int data=-1;
16
17 void setup() {
18
19     Serial.begin(9600); //시리얼 입출력을 위한 시작
20
21     for(int i=0; i<7; i++) pinMode(seg[i], OUTPUT);
22 }
```

```
24 void loop() {
25
26     if(Serial.available()>0){ //수신된 데이터 존재 여부 확인
27         data = Serial.read();
28         data -=48; //아스키코드->숫자 변환
29
30         if(0<=data && data<=9){ // 입력 데이터가 0~9 범위일 경우에
31                                     // 시리얼 모니터에서
32                                     // 출력되는 문자열
33             Serial.println("input:"+ (String)data);
34             for(int i=0; i<7; i++) digitalWrite(seg[i], 0);
35             for(int i=0; i<7; i++) digitalWrite(seg[i], numPattern[data][i]);
36         }else if(data != -38) Serial.println("wrong input");
37     }
38 }
```

// 먼저 모두 꺼주기  
// 해당 숫자 패턴 켜기  
마지막 입력 엔터는 아스키코드값 "10"

// 엔터를 제외한 다른 문자가 입력되면 출력

## 02 LCD

# LCD (Liquid Crystal Display)

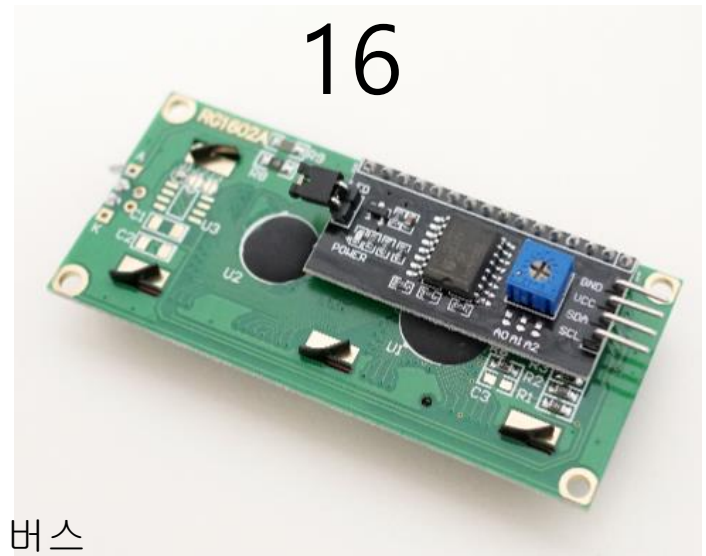
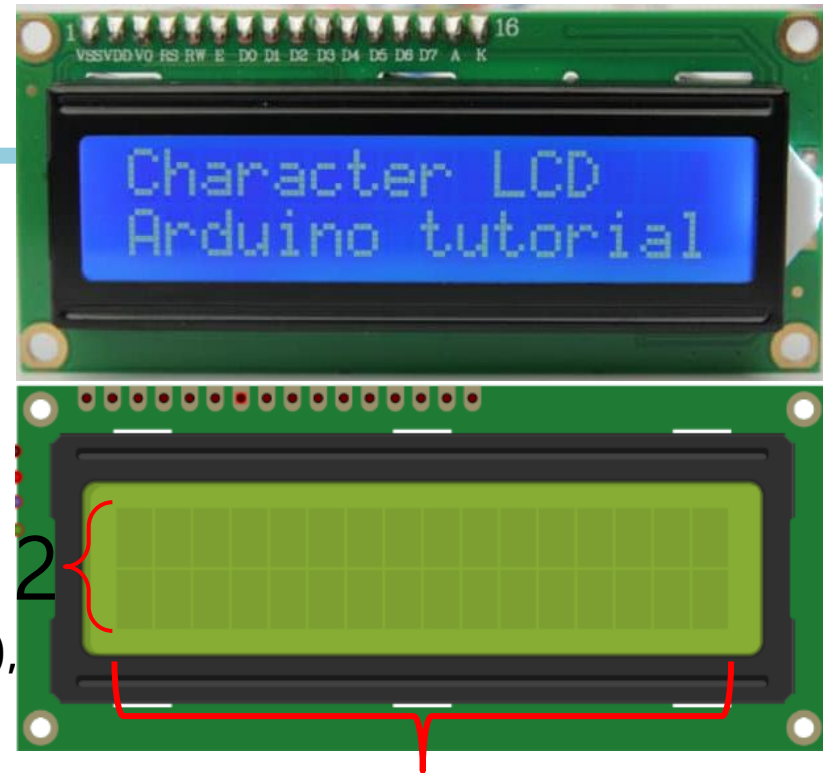
## ■ 병렬 LCD

- 16x2 크기(32문자)의 LCD를 흔히 볼 수 있음

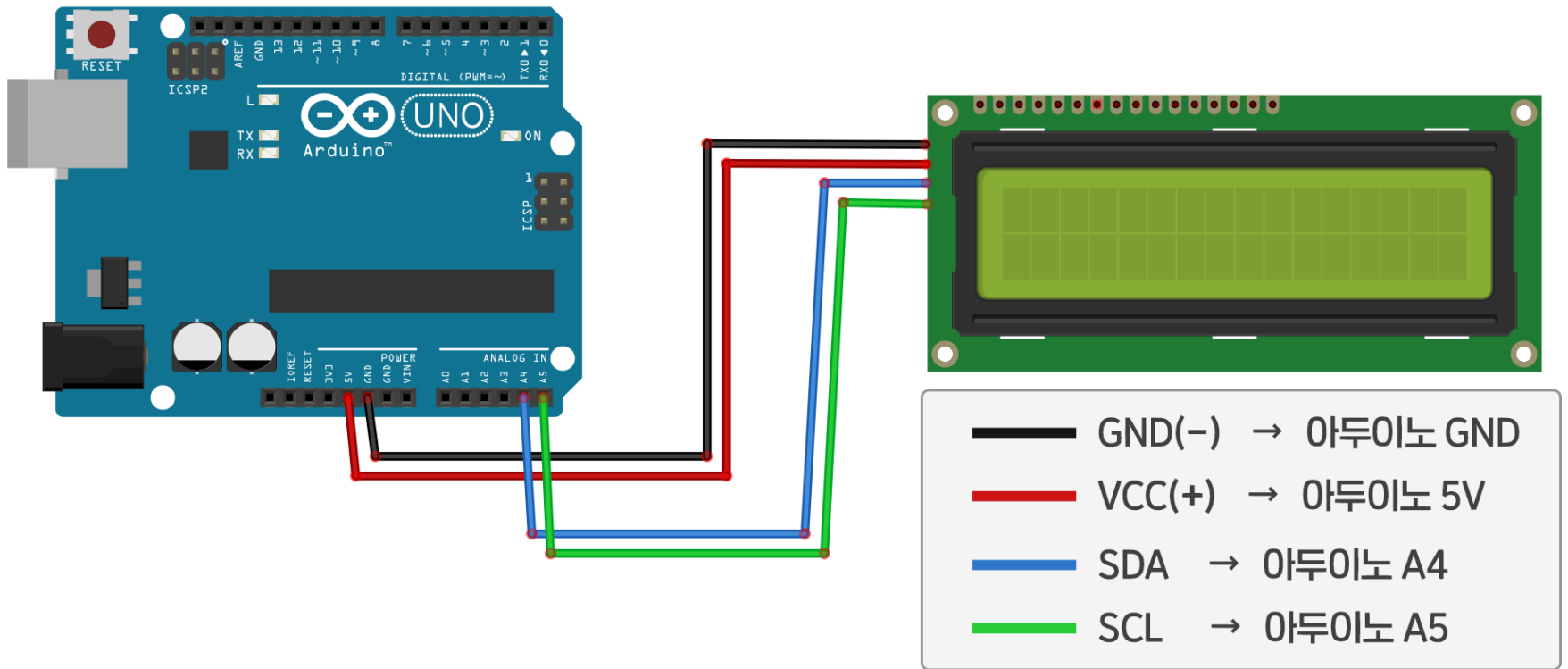
## ■ I2C LCD 모듈

- 기존 LCD에 모듈만 결합한 형태
- 4개의 핀 구성: GND, VCC, SDA(Serial Data Line), SCL(Serial Clock)
- \*I2C Converter에 가변 저항도 내장되어 있기 때문에 선명도 조절을 위한 가변 저항을 따로 추가할 필요없음
- I2C 모듈이 없다면? 기존 LCD로 아두이노와 연결하여 제어할 경우 총 6개의 digital Pin을 사용해야 하기 때문에 다양한 센서와 기타 부품을 사용할 경우에는 digital port 부족으로 사용에 한계가 있음

\* I2C (Inter-Integrated Circuit): 여러 장치 간의 통신을 하나의 버스 (2개의 신호 라인)를 통해 처리할 수 있게 하는 통신 프로토콜



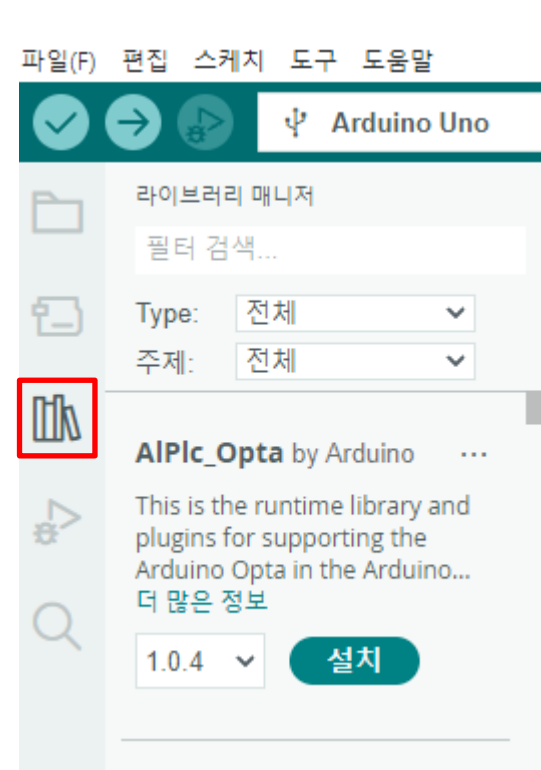
# LCD 회로



- SDA (Serial Data Line) : I2C 통신에서 데이터를 주고받는 라인(데이터 라인)
- SCL (Serial Clock Line) : I2C 통신에서 클럭 신호를 전달(클럭 라인)

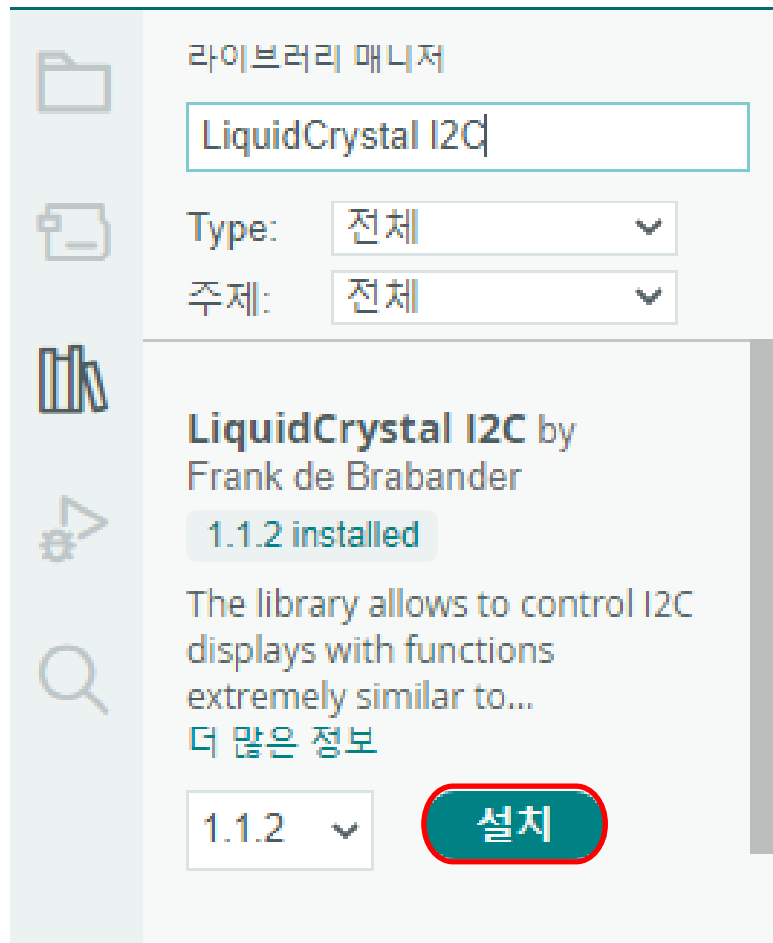
# 필요한 라이브러리 추가

- “스케치”->“라이브러리 포함”->“라이브러리 관리” 혹은 왼쪽 아이콘 클릭



# 필요한 라이브러리 추가

- “LiquidCrystal I2C” 검색 후 라이브러리 “설치”버튼 클릭



# 라이브러리 & 소스코드 경로 확인

## ■ 라이브러리 경로

| 📁 > 내 PC > 로컬 디스크 (C:) > arduino_workspace > libraries |                     |       |
|--------------------------------------------------------|---------------------|-------|
| 이름                                                     | 수정한 날짜              | 유형    |
| 📁 Arduino_JSON                                         | 2023-11-05 오후 10:43 | 파일 폴더 |
| 📁 IRremote                                             | 2024-09-21 오후 5:59  | 파일 폴더 |
| 📁 LiquidCrystal                                        | 2024-09-21 오후 5:59  | 파일 폴더 |
| 📁 LiquidCrystal_I2C                                    | 2023-08-21 오후 7:09  | 파일 폴더 |
| 📁 Pitches                                              | 2023-08-22 오후 4:37  | 파일 폴더 |
| 📁 WiFiEsp                                              | 2023-10-30 오후 6:30  | 파일 폴더 |

## ■ 소스코드 경로

| 📁 > 내 PC > 로컬 디스크 (C:) > arduino_workspace |                    |       |
|--------------------------------------------|--------------------|-------|
| 이름                                         | 수정한 날짜             | 유형    |
| 📁 HW                                       | 2023-10-03 오후 7:23 | 파일 폴더 |
| 📁 libraries                                | 2024-09-21 오후 5:59 | 파일 폴더 |
| 📁 MID                                      | 2023-10-17 오후 7:22 | 파일 폴더 |
| 📁 week02_Serial                            | 2024-09-01 오후 1:23 | 파일 폴더 |
| 📁 week03_LED_RGB                           | 2023-08-20 오후 6:17 | 파일 폴더 |
| 📁 week04_Segment_LCD                       | 2023-08-21 오후 8:26 | 파일 폴더 |

# I2C LCD의 I2C 주소 확인하기 (LCD\_I2C\_addr.ino) : 아래 코드를 입력하고 실행하면 I2C 주소가 출력됨

```
#include <Wire.h>

void setup()
{
    Wire.begin();

    Serial.begin(9600);
    while (!Serial);
    Serial.println("\nI2C Scanner");
}

void loop()
{
    byte error, address;
    int nDevices;
    Serial.println("Scanning...");
    nDevices = 0;
    for(address = 1; address < 127;
address++ )
    {
        Wire.beginTransmission(address);
        error = Wire.endTransmission();

        if (error == 0)
        {
```

```
            Serial.print("I2C device found at
address 0x");
            if (address<16)
                Serial.print("0");
            Serial.print(address,HEX);
            Serial.println("  !");
            nDevices++;
        }
        else if (error==4)
        {
            Serial.print("Unknow error at
address 0x");
            if (address<16)
                Serial.print("0");
            Serial.println(address,HEX);
        }
    }

    if (nDevices == 0)
        Serial.println("No I2C devices
found\n");
    else
        Serial.println("done\n");
    delay(5000);
}
```

# I2C LCD의 I2C 주소 확인하기 (LCD\_I2C\_addr.ino)

- 디지털 핀으로 연결하는 것이 아니기 때문에 주소 반드시 확인!

출력    시리얼 모니터    x

Message (Enter to send message to 'Arduino Uno' on 'COM3')

```
19:29:29.545 -> Scanning...
19:29:29.545 -> I2C device found at address 0x27 !
19:29:29.588 -> done
19:29:29.588 ->
19:29:34.568 -> Scanning...
19:29:34.568 -> I2C device found at address 0x27 !
19:29:34.568 -> done
19:29:34.568 ->
```

- I2C(Inter-Integrated Circuit)는 마이크로컨트롤러(아두이노 등)와 여러 장치를 연결하는 통신 규격
- 0x27은 아두이노와 LCD가 I2C 버스에서 통신할 때 사용하는 LCD의 고유 주소임.
- 이 주소를 가진 장치(LCD)와만 통신하겠다” 라고 지정하는 ID 번호 같은 개념

# 기본 함수

## ■ #include <LiquidCrystal\_I2C.h>

- 아두이노의 라이브러리 중 하나로 LCD 제어 라이브러리 포함

## ■ LiquidCrystal\_I2C lcd(0x27, 16, 2);

- LCD 제어 객체 생성 : (주소, 행, 열)

## ■ lcd.begin(16, 2);

- LCD 크기를 지정하여 초기화, 2행(y) 16열(x)

## ■ lcd.setCursor();

- 문자 표시 위치 지정

|     |     |     |     |     |     |     |     |     |     |      |      |      |      |      |      |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|------|------|------|------|------|------|
| 0,0 | 1,0 | 2,0 | 3,0 | 4,0 | 5,0 | 6,0 | 7,0 | 8,0 | 9,0 | 10,0 | 11,0 | 12,0 | 13,0 | 14,0 | 15,0 |
| 0,1 | 1,1 | 2,1 | 3,1 | 4,1 | 5,1 | 6,1 | 7,1 | 8,1 | 9,1 | 10,1 | 11,1 | 12,1 | 13,1 | 14,1 | 15,1 |

- 좌상단이 (0, 0)으로 기준점

## ■ lcd.print()

- 문자, 숫자 등 출력
- Serial.print와 기본적으로 같은 방식으로 동작

# LiquidCrystal I2C 라이브러리 함수

| 함수                                | 기능                     |
|-----------------------------------|------------------------|
| <code>begin()</code>              | LCD를 시작한다.             |
| <code>clear()</code>              | 내용을 깨끗하게 초기화(삭제)한다.    |
| <code>setCursor(col, row);</code> | col, row로 커서를 이동한다.    |
| <code>print("Codingrun")</code>   | Codingrun 글자를 출력한다.    |
| <code>home()</code>               | 커서를 (0, 0)으로 이동한다.     |
| <code>display()</code>            | 내용을 표시한다.              |
| <code>noDisplay()</code>          | 내용을 표시하지 않는다.          |
| <code>backlight()</code>          | 백라이트를 켜다.              |
| <code>noBacklight()</code>        | 백라이트를 끈다.              |
| <code>cursor()</code>             | 밑줄 커서를 표시한다.           |
| <code>noCursor()</code>           | 커서를 표시하지 않는다.          |
| <code>blink()</code>              | 깜빡이는 커서를 표시한다.         |
| <code>noBlink()</code>            | 커서를 표시하지 않는다.          |
| <code>write(ASCII 코드 값)</code>    | 한 문자를 출력한다 (ASCII 코드값) |
| <code>scrollDisplayLeft()</code>  | 표시된 내용을 좌측으로 1칸 이동한다.  |
| <code>scrollDisplayRight()</code> | 표시된 내용을 우측으로 1칸 이동한다.  |
| <code>leftToRight()</code>        | 문자 표시를 좌측에서 우측으로 설정한다. |
| <code>rightToLeft()</code>        | 문자 표시를 우측에서 좌측으로 설정한다. |

# uno\_ex05\_05: LCD 문자 출력

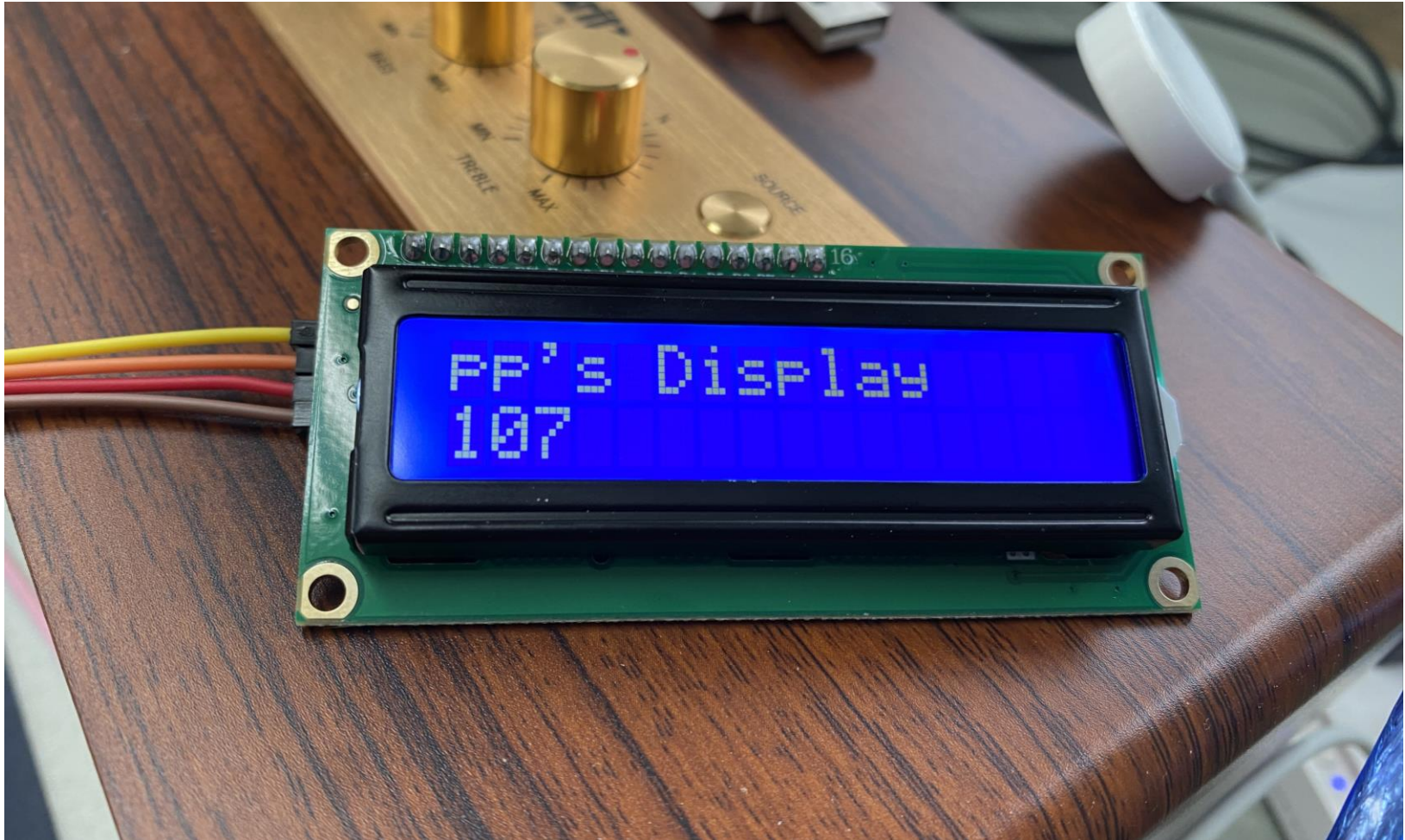
```
1  #include <LiquidCrystal_I2C.h>
2
3  LiquidCrystal_I2C lcd(0x3F ,16 ,2);
4
5  void setup()
6  {
7    lcd.begin(16,2);    // 16과 2는 아래 LCD 모듈에서 16열과 2행을 말함
8    lcd.init();         // LCD 초기화
9    lcd.backlight();    // 백라이트 온
10   lcd.clear();        // 화면 지움
11 }
12
13 void loop()
14 {
15   lcd.setCursor(5,0);  // LCD 모듈의 0번행의 5번째 열로 이동
16   lcd.print("Hi :)");
17   delay(1000);
18   lcd.setCursor(3,1);  // LCD 모듈의 1번행의 3번째 열로 이동
19   lcd.print("Coding Run");
20   delay(1000);
21   lcd.clear();
22   delay(1000);
23 }
```

|  |  |  |   |   |   |   |   |   |   |   |   |   |  |  |  |
|--|--|--|---|---|---|---|---|---|---|---|---|---|--|--|--|
|  |  |  |   |   | H | i |   | : | ) |   |   |   |  |  |  |
|  |  |  | C | o | d | i | n | g |   | R | u | n |  |  |  |

## uno\_ex05\_06: LCD 문자 출력 2

```
1  #include <LiquidCrystal_I2C.h>   주의: LiquidCrystal 라이브러리는 문자열의 길이를 확인하지 않는다.  
2                                     출력하려는 문자열이 LCD 화면 크기에 알맞은지 반드시 확인한다.  
3  // 시간 변수  
4  int time = 0;  
5  
6  // LCD 객체 생성 (I2C 주소 0x27, 16x2 LCD)  
7  LiquidCrystal_I2C lcd(0x27, 16, 2); // 0x27번 주소를 가진 16열과 2행 LCD 모듈을 I2C 통신을  
8                                     하겠다라는 의미  
9  void setup() {  
10     lcd.begin(16, 2);    // 16x2 LCD 설정  
11     lcd.init();          // 초기화  
12     lcd.backlight();     // 백라이트 켜기  
13     lcd.print("pp's Display");  
14 }  
15  
16 void loop() {  
17     // --- 첫 번째 기능: 간단한 문구 출력 ---  
18     lcd.setCursor(0, 1);    // 두 번째 줄 맨 앞  
19     lcd.print(time);  
20     delay(1000);            // 1초 대기  
21     time++;                // 시간 값 증가  
22  
23 }  
24  
25
```

# 상태바 만들기



# 사용자 정의 문자 생성하기

- 그리스 문자나 온도 표시 단위, 진행 표시 등 일반 문자로 표현할 수 없는 정보를 LCD로 출력하고 싶을 때에는 사용자 정의 문자를 생성할 수 있음
- LCD를 자세히 살펴보면 각 문자는 5x8 픽셀의 사각형 블록으로 구성
- 이 픽셀들의 값을 정의한 후 그 정보를 LCD로 보내면 사용자 정의 문자를 출력
- 문자 블록의 20%를 채운 사용자 정의 문자

```
byte p20[8]={
```

```
    B10000,
```

```
    B10000,
```

```
    B10000,
```

```
    B10000,
```

```
    B10000,
```

```
    B10000,
```

```
    B10000,
```

```
    B10000,
```

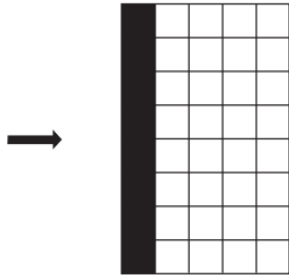
```
}
```



|   |   |   |   |   |   |
|---|---|---|---|---|---|
| B | 1 | 0 | 0 | 0 | 0 |
| B | 1 | 0 | 0 | 0 | 0 |
| B | 1 | 0 | 0 | 0 | 0 |
| B | 1 | 0 | 0 | 0 | 0 |
| B | 1 | 0 | 0 | 0 | 0 |
| B | 1 | 0 | 0 | 0 | 0 |
| B | 1 | 0 | 0 | 0 | 0 |
| B | 1 | 0 | 0 | 0 | 0 |

# 사용자 정의 문자 생성하기

```
byte p20[8]={  
    B10000,  
    B10000,  
    B10000,  
    B10000,  
    B10000,  
    B10000,  
    B10000,  
    B10000,  
}
```



- 이 코드에서는 각 문자 블록당 20%만 채우기 때문에 바이트 배열의 이름을 “p20” 지정
- 사용자 정의 문자ID는 0에서 7까지 쓸 수 있으므로 총 8개의 사용자 정의 문자 생성 가능
- Setup() 함수에서는 p20 배열을 0번 문자로 매핑하기 위해 다음과 같이 createChar() 함수 작성
  - lcd.createChar(0, p20);
- 사용자 정의 문자를 LCD로 출력할 때에는 다음과 같이 write() 함수를 사용한다.
  - lcd.write(0);

# 사용자 정의 문자 생성하기

```

7 //진행 상태 바를 나타내는 바이트 배열 선언
8 byte p20[8]={
9     B10000,
10    B10000,
11    B10000,
12    B10000,
13    B10000,
14    B10000,
15    B10000,
16    B10000,
17 };
18
19 byte p40[8]={
20    B11000,
21    B11000,
22    B11000,
23    B11000,
24    B11000,
25    B11000,
26    B11000,
27    B11000,
28 };

```

|  |  |  |  |  |
|--|--|--|--|--|
|  |  |  |  |  |
|  |  |  |  |  |
|  |  |  |  |  |
|  |  |  |  |  |
|  |  |  |  |  |
|  |  |  |  |  |
|  |  |  |  |  |
|  |  |  |  |  |

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| B | 1 | 1 | 0 | 0 | 0 |
| B | 1 | 1 | 0 | 0 | 0 |
| B | 1 | 1 | 0 | 0 | 0 |
| B | 1 | 1 | 0 | 0 | 0 |
| B | 1 | 1 | 0 | 0 | 0 |
| B | 1 | 1 | 0 | 0 | 0 |
| B | 1 | 1 | 0 | 0 | 0 |
| B | 1 | 1 | 0 | 0 | 0 |

```

29 byte p60[8]={
30     B11100,
31     B11100,
32     B11100,
33     B11100,
34     B11100,
35     B11100,
36     B11100,
37     B11100,
38 };
39
40 byte p80[8]={
41     B11110,
42     B11110,
43     B11110,
44     B11110,
45     B11110,
46     B11110,
47     B11110,
48     B11110,
49 };
50 byte p100[8]={
51     B11111,
52     B11111,
53     B11111,
54     B11111,
55     B11111,
56     B11111,
57     B11111,
58     B11111,
59 };

```

|  |  |  |
|--|--|--|
|  |  |  |
|  |  |  |
|  |  |  |
|  |  |  |
|  |  |  |
|  |  |  |
|  |  |  |
|  |  |  |

|  |  |  |
|--|--|--|
|  |  |  |
|  |  |  |
|  |  |  |
|  |  |  |
|  |  |  |
|  |  |  |
|  |  |  |
|  |  |  |

|  |  |  |
|--|--|--|
|  |  |  |
|  |  |  |
|  |  |  |
|  |  |  |
|  |  |  |
|  |  |  |
|  |  |  |
|  |  |  |



# uno\_ex05\_07: 진행 상태 바 출력하기

uno\_ex4-7.ino

```
1 //LCD 라이브러리 삽입
2 #include <LiquidCrystal_I2C.h>
3
4 //회로에 연결된 핀 번호를 사용하여 lcd 객체 인스턴스 생성
5 LiquidCrystal_I2C lcd(0x27, 16, 2);
```

```
7 //진행 상태 바를 나타내는 바이트 배열 선언
8 byte p20[8]={ 29 byte p60[8]={
9     B10000, 30     B11100,
10    B10000, 31     B11100,
11    B10000, 32     B11100,
12    B10000, 33     B11100,
13    B10000, 34     B11100,
14    B10000, 35     B11100,
15    B10000, 36     B11100,
16    B10000, 37     B11100,
17 }; 38 };
18 39
19 byte p40[8]={ 40 byte p80[8]={
20     B11000, 41     B11110,
21     B11000, 42     B11110,
22     B11000, 43     B11110,
23     B11000, 44     B11110,
24     B11000, 45     B11110,
25     B11000, 46     B11110,
26     B11000, 47     B11110,
27     B11000, 48     B11110,
28 }; 49 };
50 byte p100[8]={
51     B11111,
52     B11111,
53     B11111,
54     B11111,
55     B11111,
56     B11111,
57     B11111,
58     B11111,
59 };
```

# uno\_ex05\_07: 진행 상태 바 출력하기

```
61 void setup()
62 {
63     //LCD의 행과 열을 16행 2열로 설정
64     lcd.begin(16, 2);
65
66     lcd.init(); //초기화
67     lcd.backlight(); //배경 라이트 on
68
69     //진행 상태 바 만들기
70     lcd.createChar(0, p20);
71     lcd.createChar(1, p40);
72     lcd.createChar(2, p60);
73     lcd.createChar(3, p80);
74     lcd.createChar(4, p100);
75 }
```

|   | 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|---|
| B | 1 | 1 | 0 | 0 | 0 |
| B | 1 | 1 | 0 | 0 | 0 |
| B | 1 | 1 | 0 | 0 | 0 |
| B | 1 | 1 | 0 | 0 | 0 |
| B | 1 | 1 | 0 | 0 | 0 |
| B | 1 | 1 | 0 | 0 | 0 |
| B | 1 | 1 | 0 | 0 | 0 |

- 사용자 정의 문자(5×8 도트 패턴)를 LCD의 “슬롯”에 저장하는 함수
- 이 함수 자체가 화면에 뭔가를 출력하지는 않음

```
77 void loop()
78 {
79     lcd.clear();
80     //LCD에 문자열 출력
81     lcd.print("pp's Display");
82
83     //커서를 2열 1행으로 이동
84     lcd.setCursor(0,1);
85
86     //2열에서 진행 상태 바를 만들려고 16문자만큼 반복
87     for(int i=0; i<16; i++)
88     {
89         //각 문자마다 다섯 번 반복하여 진행 상태 바를 만들
90         for(int j=0; j<5; j++)
91         {
92             lcd.setCursor(i, 1); //i행으로 커서 이동
93             lcd.write(j); //상태 바 업데이트
94             delay(100); //0.1초간 대기
95         }
96     }
97
98 }
```

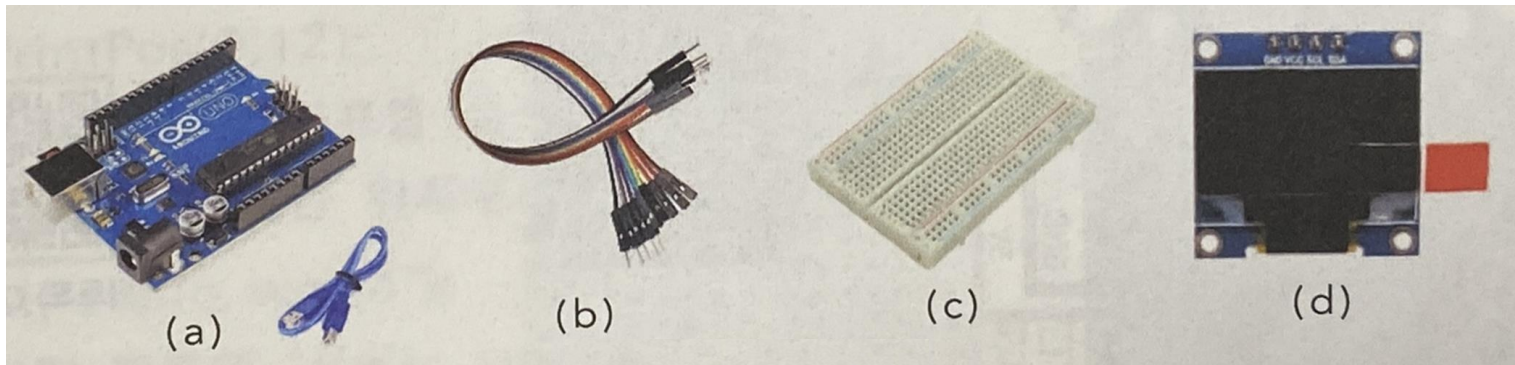
- 저장해둔 사용자 정의 문자를 LCD에 출력하는 함수

### 3. 0.96인치 OLED 디스플레이

#### ■ 개요

- 이번 장에서는 우노 R3를 이용하여 0.96인치 OLED 디스플레이 제어에 대해 알아보자.
- OLED 디스플레이에 "Hellow World"를 출력, 간단한 도형 및 애니메이션 구현 예제임.

- 사용 디바이스 : 우노 R3(a), 점퍼 케이블(b), 브레드 보드(c), 0.96인치 OLED(d)



### 3. 0.96인치 OLED 디스플레이

#### ■ 배경 지식

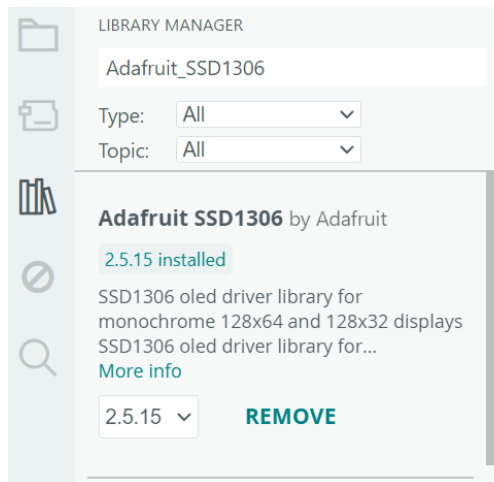
- OLED OLED는 Organic Light Emitting Diode의 약자임. 즉, 유기발광다이오드. 일반 LED와 달리 자체 발광이 가능하므로 백라이트가 필요없음
- OLED 0.96 인치 디스플레이 모듈 웨어러블 또는 IoT를 위한 OLED로 0.96 인치의 초소형 사이즈 디스플레이 모듈임.
- OLED LCD는 아래 표와 같은 특징을 갖으며 저전력으로 동작해 웨어러블 기기에 적합함. 또한 128x64 해상도를 갖고 있어 글자뿐 아니라 이미지 표현이 가능함.

| 항목       | 내용               |
|----------|------------------|
| 동작 전압    | 3.3V             |
| 해상도      | 128 × 64         |
| 드라이버 IC  | SSD1306          |
| 통신 인터페이스 | I <sup>2</sup> C |
| 크기       | 27 × 27 × 4.1 mm |

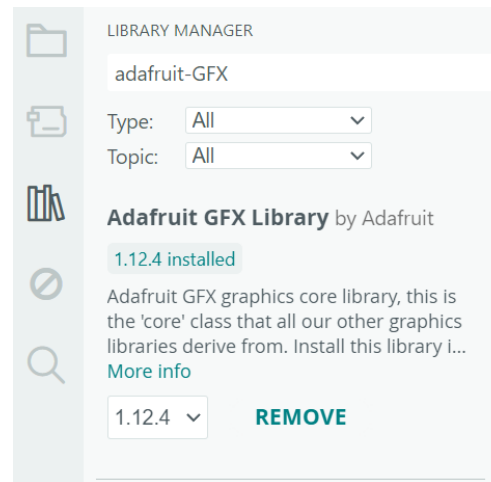
### 3. 0.96인치 OLED 디스플레이

#### ■ 라이브러리 다운로드 및 설치하기

- OLED를 사용하기 위해서는 라이브러리와 Bitmap 파일을 Hex(16진수) 행렬값으로 변환해주는 일 2가지를 다운 받아야 함.
- 단, 글씨만 디스플레이 하려면 라이브러리 파일만 다운로드 받으면 됨.
- 아두이노 라이브러리 설치과정은 아두이노 IDE에서 스케치 - 라이브러리 포함하기 - 라이브러리 관리에서 Adafruit\_SSD1306/ Adafruit-GFX-Library를 선택하여 다운로드 함.



```
Code.ino
1  #include "Adafruit_SSD1306.h"
2
3  // I2C 방식의 OLED
4  U8GLIB_SSD1306_128 u8g(U2, U3, U4);
5
6  void setup() {
7    // 아무
8  }
9
10 void loop() {
11
12     u8g.firstPage();
13     do {
14
15         // 텍스트 출력
```

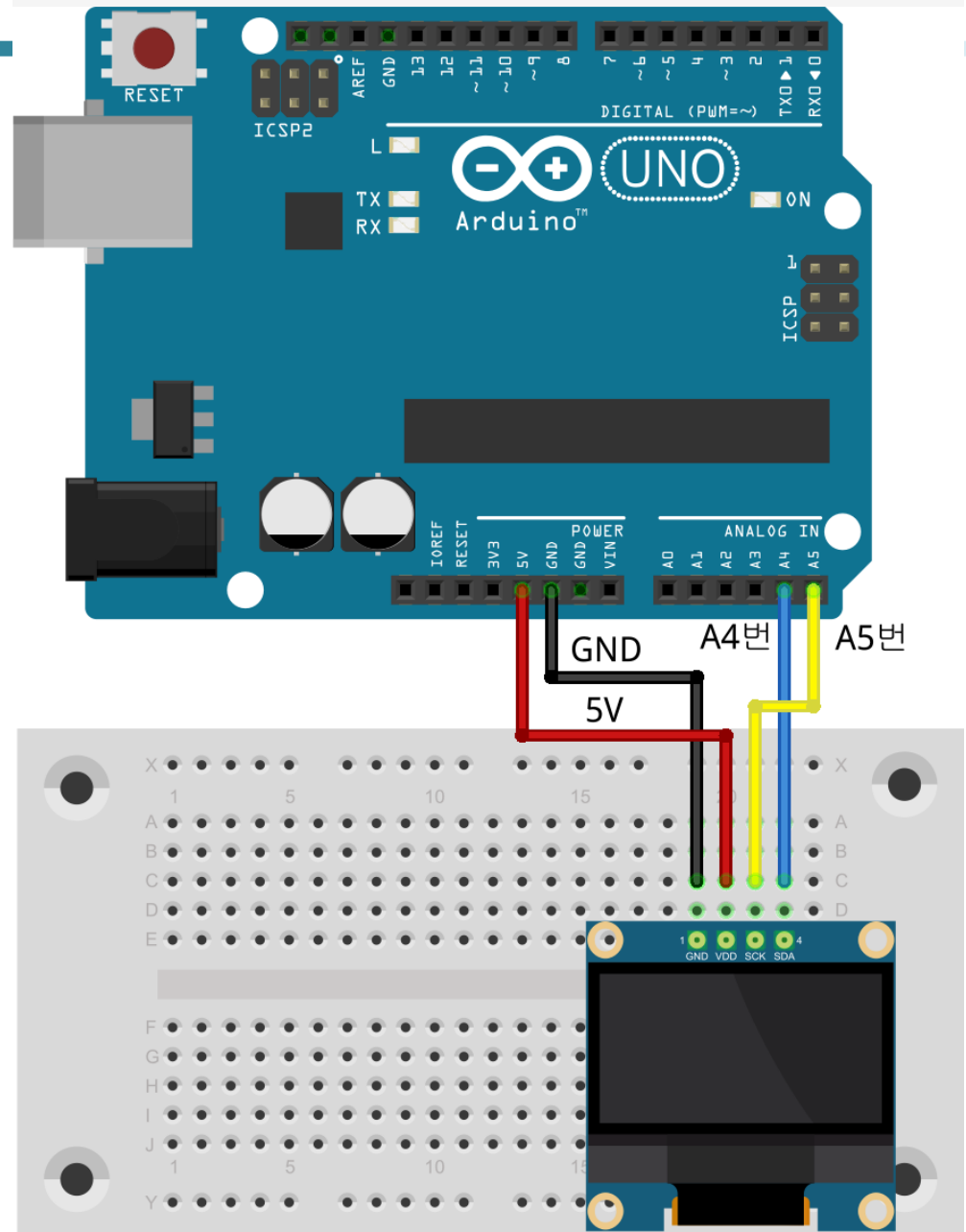


```
Code.ino
1  #include "U8glib.h"
2
3  // I2C 방식의 OLED
4  U8GLIB_SSD1306_128 u8g(U2, U3, U4);
5
6  void setup() {
7    // 아무 설정이 필요
8  }
9
10 void loop() {
11
12     u8g.firstPage();
13     do {
14
15         // 텍스트 출력
```

### 3. 0.96인치 OLED 디스플레이

#### ■ 구현 회로도

| 우노 R3   | OLED |
|---------|------|
| 아날로그 4번 | SDA  |
| 아날로그 5번 | SCL  |
| 5V      | VCC  |
| GND     | GND  |



### 3. 0.96인치 OLED 디스플레이 - 코드 함수 설명

#### ■ 함수 설명

- `u8g.firstPage()` : 화면의 첫 번째 페이지를 초기화한다. U8glib 은 페이지 모드에서 디스플레이를 업데이트를 하여 이 함수를 이용해 페이지의 시작을 알린다.
- `do (... while (u8g.nextPage()))` : OLED 디스플레이의 모든 페이지를 순차적으로 그리는 함수 루프이다. do-while 구조 안에서 페이지에 그릴 내용을 정의하고, `u8g.nextPage()`를 호출해 다음 페이지로 넘어간다. 마지막 페이지까지 그리면 루프가 종료된다.
- `u8g.setFont(u8g_font_unifont)` : OLED 디스플레이에 출력될 문자의 폰트를 설정한다. 이 코드에서는 미리 정의된 `u8g_font_unifont` 폰트를 사용한다.
- `u8g.setPrintPos(0,12)` : 출력할 문자의 시작 좌표를 설정한다. 여기서는 (0, 12) 좌표로 설정하여, 왼쪽 상단에 서 조금 아래로 내려간 위치에서 문자가 출력되도록 한다.
- `u8g.print("Hello, world")` : 현재 설정된 좌표에 "Hello, world" 문자열을 출력한다.
- `u8g.drawStr(0,32, "Hello, world!")` : 특정 좌표 (0, 32)에 "Hello, world!"라는 문자열을 출력한다. `u8g.print()`와 유사하지만, `drawStr` 함수는 좌표와 문자열을 한 번에 지정하여 출력한다.

### 3. 0.96인치 OLED 디스플레이 - 텍스트 출력 및 간단한 도형 그리기

```
#include "U8glib.h"
// I2C 방식의 OLED 사용 시 (SSD1306 128x64 기준)
U8GLIB_SSD1306_128X64 u8g(U8G_I2C_OPT_NONE); //초기화

void setup() {
    // 아무 설정이 필요 없어도 OK
}

void loop() {

    u8g.firstPage(); //첫 페이지 초기화
    do {

        // ① 텍스트 출력
        u8g.setFont(u8g_font_unifont); // 폰트 설정, 미리 정의된 u8g_font_unifont 폰트를 사용
        u8g.setPrintPos(0, 14); //출력할 문자의 시작 좌표를 설정
        u8g.print("Hello, OLED!"); // 현재 설정된 좌표에 "Hello, world" 문자열을 출력

        u8g.drawStr(0, 30, "Text + Drawing"); //특정 좌표 (0, 32)에 "Hello, world!"라는 문자열을 출력

        // ② 사각형 그리기
        u8g.drawFrame(0, 40, 40, 20); // 외곽 사각형
        u8g.drawBox(45, 40, 20, 20); // 깍 찬 사각형

        // ③ 원 그리기
        u8g.drawCircle(90, 50, 10); // 중심 (90,50), 반지름 10

        // ④ 직선(line) 그리기
        //u8g.drawLine(0, 0, 127, 63); // 대각선 직선

        // ⑤ 점(pixel) 찍기
        u8g.drawPixel(120, 10);

    } while (u8g.nextPage());
    delay(500);
}
```

**u8g.drawFrame(x, y, w, h);**

x: 사각형 왼쪽 위 모서리의 x좌표 (0~127)

y: 사각형 왼쪽 위 모서리의 y좌표 (0~63)

w: 사각형 가로 길이 (픽셀 단위)

h: 사각형 세로 길이 (픽셀 단위)

**u8g.drawBox(x, y, w, h);**

x: 사각형 왼쪽 위 모서리의 x좌표 (0~127)

y: 사각형 왼쪽 위 모서리의 y좌표 (0~63)

w: 사각형 가로 길이 (픽셀 단위)

h: 사각형 세로 길이 (픽셀 단위)

**u8g.drawCircle(x, y, r);**

x: 원 중심 x좌표

y: 원 중심 y좌표

r: 반지름 (픽셀 단위)

**u8g.drawPixel(x, y);**

x: 점의 x좌표

y: 점의 y좌표

### 3. 0.96인치 OLED 디스플레이 실습 예제 - 애니메이션

```
#include "U8glib.h"

U8GLIB_SSD1306_128X64 u8g(U8G_I2C_OPT_NONE);

int x = 0;           // 공의 x좌표
int dir = 1;         // 방향 (1=오른쪽, -1=왼쪽)

void setup(){}

void loop(){
  u8g.firstPage();
  do {
    // 공 그리기
    u8g.drawDisc(x, 30, 8); // (x,30) 위치에 반지름 8짜리 공
  } while (u8g.nextPage());

  // 좌표 이동
  x += dir;

  // 화면 끝에 닿으면 방향 전환
  if (x > 120) dir = -1;
  if (x < 0) dir = 1;

  delay(10); // 프레임 속도
}
```

# 요약

- 세그먼트 LED
  - 막대 하나 당 디지털 출력
  - 선과 핀 번호 정확하게 매칭
- LCD
  - 16X2 문자 출력
  - 문자 한 개 당 5X8 데이터 출력
  - 핀 번호가 아닌 출력 버스 주소 확인
  - print함수 등 LiquidCrystal 라이브러리에 정의된 함수 사용

**Thank You**